

Electronics Repair Shop Management System

Technical Specification

Technical blueprint for a modular, future-proof management system. The first implementation targets an electronics repair shop while keeping the core platform reusable for future business modules.

1. Technical Objectives

- Provide a reliable system of record for customers, devices, repairs, inventory, sales, and accounting.
- Use a modular architecture that allows features to be added without tightly coupling modules.
- Maintain transactional consistency for financial and inventory operations.
- Provide complete auditability for important business changes.
- Expose reusable APIs for web, mobile, integrations, and future customer portals.
- Support single-branch deployment initially while allowing multi-branch expansion.

2. Recommended Technology Stack

Layer	Recommendation	Purpose
Frontend	Next.js + TypeScript	Admin dashboard, POS, repair screens, reporting, settings.
UI	React + Tailwind CSS	Reusable responsive interface.
Backend	ASP.NET Core / C#	Business logic, APIs, authentication integration, background jobs.
API	REST + optional GraphQL	REST for transactional APIs; GraphQL for complex read/query screens if needed.
Database	PostgreSQL or SQL Server	Primary relational data store.
Cache	Redis	Caching, rate limiting, distributed locks, short-lived state.
File Storage	S3-compatible object storage	Device photos, documents, receipts, attachments.
Background Jobs	Hangfire / hosted workers	Notifications, scheduled reports, recurring tasks.
Auth	OAuth/OIDC-compatible identity	Authentication, sessions, MFA, role-based authorization.
Deployment	Docker + cloud hosting	Consistent environments and scalable deployment.
Monitoring	Structured logs + error monitoring	Operational visibility and troubleshooting.

3. High-Level Architecture

Use a modular monolith initially. Modules share a database but have clear boundaries, application services, domain models, and repositories. Domain events allow cross-module automation without direct coupling.

Client → API → Application Modules → Domain Layer → Persistence / External Services

- Identity and Access
- Customer Management
- Device Management
- Repair Management
- Service Catalog
- Inventory
- Purchasing
- Sales / POS
- Accounting
- Employees
- Notifications

- Reporting
- Audit

4. Module Boundaries

Module	Owns	Key Dependencies
Identity	Users, roles, permissions, sessions	All modules
CRM	Customers, contacts	Devices, repairs, sales
Devices	Device records, identifiers, history	Customers, repairs
Repairs	Repair tickets, diagnosis, status, services	Customers, devices, inventory, sales
Services	Repair/service catalog and pricing	Repairs
Inventory	Items, stock ledger, locations	Repairs, sales, purchasing
Purchasing	Suppliers, POs, receipts	Inventory, accounting
Sales	Sales, invoices, payments	Inventory, CRM, accounting
Accounting	Accounts, journals, ledger	Sales, purchasing, expenses, repairs
Reporting	Read models and aggregates	All modules
Notifications	Templates, delivery jobs	Events from all modules
Audit	Change history	All modules

5. Core Data Model

Entity	Important Fields
Organization	id, name, business_info, status, created_at
Branch	id, organization_id, name, address, status
User	id, organization_id, employee_id, email, status
Role / Permission	role_id, permission_code, scope
Customer	id, organization_id, name, contact, address, notes
Device	id, customer_id, type, brand, model, serial_no, imei, condition
Repair	id, branch_id, customer_id, device_id, status, diagnosis, dates, totals
RepairService	id, repair_id, service_id, quantity, price, discount
Service	id, category, name, default_price, warranty_days, status
InventoryItem	id, sku, name, category, cost, selling_price, min_stock
StockMovement	id, item_id, branch_id, type, quantity, unit_cost, reference
Supplier	id, organization_id, name, contacts, terms
PurchaseOrder	id, supplier_id, branch_id, status, totals
Sale	id, branch_id, customer_id, status, totals
Payment	id, transaction_type, transaction_id, method, amount, status
Account	id, organization_id, code, name, type
JournalEntry	id, date, source_type, source_id, status
JournalLine	id, journal_id, account_id, debit, credit
Expense	id, branch_id, category, amount, date, payment_id
AuditLog	id, actor_id, action, entity_type, entity_id, before, after, timestamp

6. Repair Workflow

Default states should be configurable rather than hard-coded.

- RECEIVED
- DIAGNOSIS
- WAITING_FOR_APPROVAL
- APPROVED
- WAITING_FOR_PARTS
- REPAIRING
- TESTING
- READY_FOR_PICKUP
- COMPLETED
- CANCELLED

Every state transition should record actor, timestamp, previous status, new status, and optional reason.

7. Inventory Ledger Design

Inventory quantity should be derived from a transaction ledger rather than relying only on a mutable quantity field.

- PURCHASE_RECEIPT
- REPAIR_CONSUMPTION
- SALE

- CUSTOMER_RETURN
- SUPPLIER_RETURN
- DAMAGE
- ADJUSTMENT
- TRANSFER

Each movement must reference its source transaction and retain quantity, unit cost, branch/location, actor, and timestamp.

8. Accounting Engine

Use double-entry accounting for the financial core. Business modules create accounting events; the accounting module converts approved events into journal entries.

- Every journal entry must balance: total debits = total credits.
- Journal entries should reference their source transaction.
- Posted accounting entries should not be silently edited; corrections should use reversing or adjusting entries.
- Maintain account codes and account types in a configurable chart of accounts.
- Support cash, bank, receivables, payables, inventory, revenue, COGS, and expense accounts.

9. Domain Events

Event	Possible Consumers
RepairCreated	Notifications, reporting
RepairCompleted	Inventory, accounting, notifications, reporting
PartConsumed	Inventory, accounting, reporting
SaleCompleted	Inventory, accounting, reporting
PaymentReceived	Accounting, notifications, reporting
PurchaseReceived	Inventory, accounting
ExpenseRecorded	Accounting, reporting
StockBelowMinimum	Notifications, purchasing

10. API Design

- Use versioned APIs, for example `/api/v1/`.
- Use consistent resource naming and HTTP status codes.
- Validate all input at the API boundary.
- Enforce organization/branch authorization server-side.
- Use idempotency keys for payment and other retry-sensitive operations.
- Return stable error codes for frontend handling.
- Use pagination, filtering, sorting, and search for list endpoints.
- Keep internal database identifiers out of user-controlled authorization decisions.

11. Representative API Endpoints

Area	Examples
Customers	GET <code>/api/v1/customers</code> , POST <code>/api/v1/customers</code>
Devices	GET <code>/api/v1/customers/{id}/devices</code> , POST <code>/api/v1/devices</code>
Repairs	POST <code>/api/v1/repairs</code> , GET <code>/api/v1/repairs/{id}</code> , PATCH <code>/api/v1/repairs/{id}/status</code>
Services	GET <code>/api/v1/services</code> , POST <code>/api/v1/services</code>
Inventory	GET <code>/api/v1/inventory/items</code> , POST <code>/api/v1/inventory/movements</code>
Purchasing	POST <code>/api/v1/purchase-orders</code> , POST <code>/api/v1/purchase-orders/{id}/receive</code>
Sales	POST <code>/api/v1/sales</code> , POST <code>/api/v1/sales/{id}/payments</code>
Accounting	GET <code>/api/v1/accounts</code> , GET <code>/api/v1/journal-entries</code>
Reports	GET <code>/api/v1/reports/sales</code> , <code>/repairs</code> , <code>/inventory</code> , <code>/financial</code>

12. Security Requirements

- Password hashing must use a modern adaptive password hash if local passwords are supported.
- Use secure, HTTP-only cookies or an equivalent secure token strategy.
- Implement role-based and organization/branch-scoped authorization.
- Validate file uploads and restrict file types and sizes.
- Encrypt sensitive data in transit and protect secrets outside source control.
- Rate-limit authentication and sensitive endpoints.
- Record security-sensitive actions in the audit log.
- Use least-privilege database and service accounts.
- Provide backup, recovery, and retention procedures.

13. Transaction and Concurrency Requirements

- Repair completion, stock deduction, and payment operations must use database transactions where consistency requires it.
- Prevent negative inventory where the business policy disallows it.
- Use row/version locking or optimistic concurrency for stock and other high-conflict records.
- Prevent duplicate payments and duplicate event processing through idempotency and unique constraints.
- Use database constraints for critical invariants rather than relying only on frontend validation.

14. Audit Requirements

- Record create, update, delete, approve, cancel, post, reverse, and permission-change actions.
- Store actor, organization, branch, timestamp, entity, action, and relevant before/after values.

- Do not allow normal users to modify audit records.
- Provide searchable audit history to authorized administrators.

15. Reporting Architecture

Operational reports may query transactional data directly at small scale. As usage grows, create reporting read models or materialized aggregates so complex reports do not slow transactional operations.

- Daily sales
- Repair volume and turnaround time
- Inventory valuation and movement
- Revenue and expenses
- Accounts receivable and payable
- Profit and loss
- Branch performance
- Service and device analytics

16. File and Photo Storage

- Store device photos, receipts, documents, and attachments in object storage rather than database blobs where practical.
- Store metadata and ownership references in the database.
- Use private storage by default and issue time-limited access URLs when files are requested.
- Record uploaded-by and uploaded-at metadata.

17. Notifications and Background Jobs

- Use asynchronous jobs for email, SMS, scheduled reports, reminders, and other non-transactional work.
- Use retry policies with bounded attempts.
- Track delivery status and failure reason.
- Keep notification templates configurable.
- Do not block a repair or payment transaction because an external notification provider is unavailable.

18. Search

- Global search should support repair number, customer, phone number, device model, serial number, IMEI, invoice number, and SKU.
- Use indexed database search for the initial implementation.
- Introduce a dedicated search engine only if data volume or search requirements justify it.

19. Testing Strategy

- Unit tests for domain rules and calculations.
- Integration tests for database transactions and module boundaries.
- API tests for authentication, authorization, validation, and business workflows.
- End-to-end tests for critical flows such as repair completion, inventory consumption, sales, and payment.
- Accounting tests must verify that every generated journal entry balances.
- Regression tests for inventory and payment idempotency.

20. Backup and Recovery

- Automated database backups.
- Encrypted backup storage.
- Defined retention policy.
- Periodic restore testing.
- Documented recovery procedure.
- Separate production credentials from development and test environments.

21. Deployment Environments

Environment	Purpose
Development	Local development and rapid testing.
Staging	Production-like validation and acceptance testing.
Production	Live business operations.

22. Non-Functional Requirements

Area	Requirement
Performance	Normal CRUD operations should feel responsive; use pagination and indexes for large lists.

Area	Requirement
Availability	Production should use monitored managed infrastructure and automated backups.
Scalability	Application should support additional branches and users without redesigning the data model.
Security	Server-side authorization, secure secrets, validation, audit logging, and least privilege.
Maintainability	Clear module boundaries, typed contracts, automated tests, documentation.
Observability	Structured logs, error monitoring, health checks, and basic metrics.
Data Integrity	Transactions, constraints, unique keys, idempotency, and accounting balance checks.

23. Implementation Priority

Phase 1 — Identity, organization/branch, customers, devices, repairs, services, audit logs.

Phase 2 — Inventory, suppliers, purchasing, stock ledger, low-stock alerts.

Phase 3 — POS, invoices, payments, receipts, returns.

Phase 4 — Accounting engine, chart of accounts, journals, ledger, financial reports.

Phase 5 — Notifications, scheduled jobs, customer portal, advanced reporting.

Phase 6 — Multi-branch, mobile technician app, integrations, advanced analytics, AI features.

24. Design Constraints and Notes

- Do not start with a large microservice architecture unless scale or organizational requirements justify it.
- Do not make accounting a collection of manually editable totals; use ledger-based transactions.
- Do not treat inventory quantity as the only source of truth; retain movements.
- Do not hard-code repair statuses, service fees, roles, or business settings.
- Keep customer-facing pricing separate from internal costs.
- Tax and accounting compliance for Philippine operations should be reviewed by a qualified accountant before production use.